

Bases de Datos

Clase 16: Algoritmos de Join (II)

Sort-Merge Join · Hash Join

Recordatorio: lo que ya sabemos

Algoritmo	Costo	Limitación
NLJ	$\text{Pág}(R) + \text{Tuplas}(R) \cdot \text{Pág}(S)$	Muy lento
BNLJ	$\text{Pág}(R) + \lceil \text{Pág}(R)/(B-2) \rceil \cdot \text{Pág}(S)$	Depende mucho del buffer
INLJ	$\text{Pág}(R) + \text{Tuplas}(R) \cdot \text{CostoBúsq}(S)$	Requiere índice en S

Hoy: dos algoritmos que no requieren índice y son **mucho más eficientes** que NLJ/BNLJ para tablas grandes.

Sort-Merge Join

Sort-Merge Join (SMJ): idea

Dos fases:

1. **Sort:** ordenar R y S por el atributo de join (External Merge Sort)
2. **Merge:** recorrer ambas relaciones ordenadas **en paralelo**, emitiendo matches

SMJ: la fase de merge

Ambas relaciones están ordenadas por el atributo de join.

				pR			
				↓			
R (ordenada por a):	1,	3,	5,	5,	7,	9,	...
S (ordenada por a):	2,	3,	5,	6,	7,	7,	...
				↑			
				pS			

- Si $R[pR] < S[pS] \rightarrow$ avanzar pR
- Si $R[pR] > S[pS] \rightarrow$ avanzar pS
- Si $R[pR] = S[pS] \rightarrow$ **match** $\rightarrow$ emitir todas las combinaciones, luego avanzar

SMJ: merge (caso simple)

```
pR := primera tupla de R (ordenada)
pS := primera tupla de S (ordenada)

while pR ≠ null AND pS ≠ null do
  if pR.a < pS.a then pR := siguiente en R
  else if pR.a > pS.a then pS := siguiente en S
  else // pR.a = pS.a → MATCH
    emitir (pR, pS) y avanzar
```

SMJ: manejo de duplicados

Cuando varias tuplas comparten el mismo valor de join, necesitamos emitir **todas las combinaciones**. La idea: **marcar** el inicio del grupo en S y "rebobinar" por cada tupla de R que siga en el grupo.

```
else // pR.a = pS.a → MATCH: inicio de un grupo
    mark := pS // recordar inicio del grupo en S
    while pR ≠ null and pR.a = mark.a:
        pS := mark // rebobinar al inicio del grupo
        while pS ≠ null and pS.a = pR.a:
            emitir (pR, pS)
            pS := siguiente en S
        pR := siguiente en R
```

SMJ: costo

$$\text{Costo SMJ} = \text{Sort}(R) + \text{Sort}(S) + \text{Páginas}(R) + \text{Páginas}(S)$$

Donde $\text{Sort}(X) \approx 4 \cdot \text{Páginas}(X)$ **sin pipelining** (2 fases de External Merge Sort).

$$\text{Costo SMJ (sin pipelining)} \approx 5 \cdot (\text{Pág}(R) + \text{Pág}(S))$$

Esto se reduce un poco con pipelining, vamos a ver algo de esto cuando veamos external sorting.

SMJ: ejemplo numérico

R y S de 16 MB cada una (2.048 páginas cada una).

$$\text{Costo} \approx 5 \times (2.048 + 2.048) = 20.480 \text{ I/O}$$

A 0,1 ms por I/O $\rightarrow \approx$ **2 segundos**

Si R ya está ordenada (nos ahorramos $\text{Sort}(R) = 4 \cdot \text{Pág}(R)$):

- R ya ordenada $\rightarrow$ se lee **una sola vez** en el merge: **$\text{Pág}(R)$**
- S se ordena (**$4 \cdot \text{Pág}(S)$**) y se re-lee en el merge: **$+\text{Pág}(S)$**

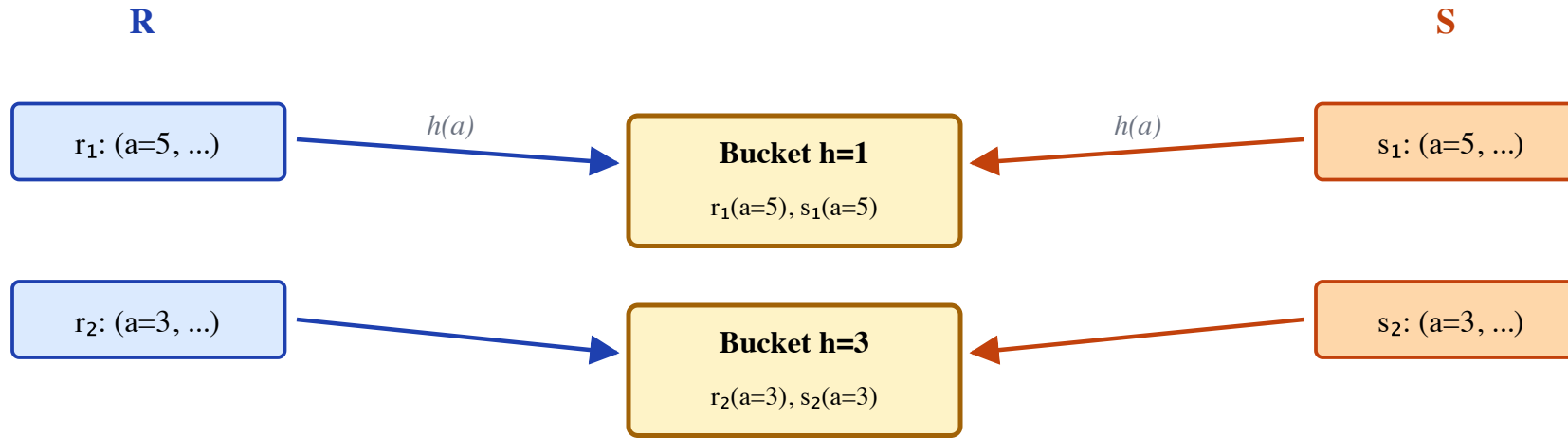
$$\text{Costo} \approx \text{Pág}(R) + 4 \cdot \text{Pág}(S) + \text{Pág}(S) = 2.048 + 10.240 = 12.288 \text{ I/O} \approx 1,2 \text{ seg}$$

SMJ: cuándo conviene

- Cuando las tablas son **grandes** y no hay índice
- O si una o ambas tablas ya están **ordenadas** por el atributo de join
- El resultado sale **ordenado** → útil si la consulta tiene `ORDER BY`
- Funciona bien con tablas de tamaños **muy diferentes** (a diferencia de Hash Join, que viene a continuación)

Hash Join

Hash Join (HJ): idea



$$R.a = S.a \Rightarrow h(R.a) = h(S.a) \Rightarrow \text{mismo bucket}$$

Usa **hashing** para evitar comparar todas las tuplas de R contra todas las de S.

Propiedad clave: si $R.a = S.a$, entonces $h(R.a) = h(S.a) \rightarrow$ ambas caen en el mismo bucket.

Solo comparamos tuplas dentro del mismo bucket — el hash descarta el resto sin mirarlo.

Hash Join (HJ): idea

Dos escenarios según cuánta memoria tengamos:

- **Caso 1 — In-memory HJ:** la tabla más chica cabe en el buffer
- **Caso 2 — Grace HJ:** no cabe → particionamos primero

In-memory HJ: BUILD + PROBE

Supongamos que **toda R cabe en el buffer** (R es la más chica).

```
// Fase BUILD – construir la hash table con R
hash_table := vacía
for each tupla r ∈ R:
    hash_table.insert(h(r.a), r)

// Fase PROBE – recorrer S y buscar matches en la hash table
for each tupla s ∈ S:
    for each r en hash_table.lookup(h(s.a)):
        if r.a = s.a then                // verificar por colisiones
            emitir (r, s)
```

In-memory HJ: costo

- **BUILD:** leemos R entera, la hash table vive en RAM $\rightarrow \text{Pág}(R)$ I/O
- **PROBE:** leemos S entera, cada lookup es en RAM (0 I/O) $\rightarrow \text{Pág}(S)$ I/O

$$\text{Costo HJ in-memory} = \text{Pág}(R) + \text{Pág}(S)$$

Es el **mínimo teórico** de un join: cada página se lee exactamente una vez.

Requiere: $\text{Pág}(R) \leq B - 2$ (R cabe en el buffer, dejando una página para leer S y una para output).

¿Y si R no cabe en RAM?

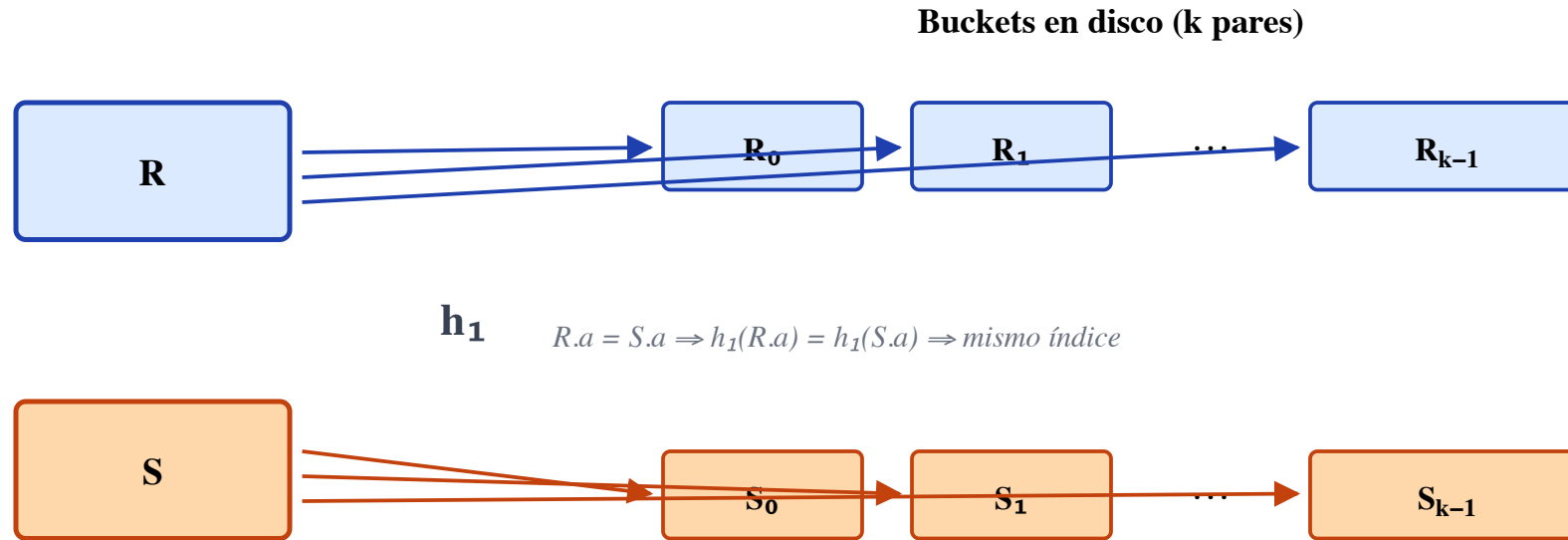
Si $\text{Pág}(R) > B - 2$, no podemos armar una sola hash table (en la práctica el umbral es algo menor por el factor de carga de la tabla).

Idea: particionar ambas tablas primero con una función h_1 .

- Obtenemos k pares de buckets: $(R_0, S_0), (R_1, S_1), \dots, (R_{\{k-1\}}, S_{\{k-1\}})$
- Cada par es un **join independiente más chico**
- Si cada R_i cabe en RAM $\rightarrow$ aplicamos **in-memory HJ** a cada par

Este algoritmo se llama **Grace Hash Join** (o "HJ particionado").

Grace HJ: Fase 1 — Partición

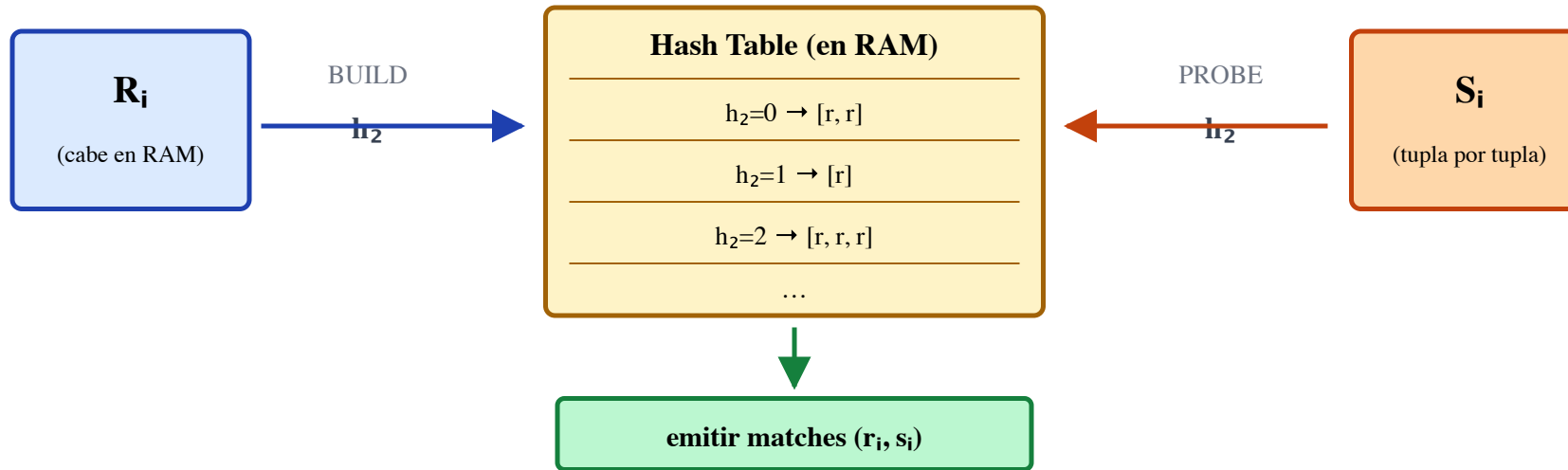


- Leemos cada tupla, la hasheamos, la **escribimos** al bucket correspondiente
- **Costo Fase 1:** leer R + escribir R + leer S + escribir S = $2 \cdot (\text{Pág}(R) + \text{Pág}(S))$

Como h_1 respeta la propiedad clave, los matches caen al **mismo índice** de bucket.

Grace HJ: Fase 2 — Probe por bucket

Procesando el par (R_i, S_i)



Para cada $i = 0, 1, \dots, k-1$:

1. Cargar R_i (ya chico) a una **hash table en RAM** usando h_2 (distinta de h_1)
2. Leer S_i tupla por tupla, probar en la hash table, emitir matches

Cada bucket es un **in-memory HJ independiente** aplicado al par (R_i, S_i) .

Grace HJ: costo total

$$\begin{aligned}\text{Costo Grace HJ} &= \underbrace{2 \cdot (\text{Pág}(R) + \text{Pág}(S))}_{\text{Fase 1: partición}} + \underbrace{\text{Pág}(R) + \text{Pág}(S)}_{\text{Fase 2: probe}} \\ &= 3 \cdot (\text{Pág}(R) + \text{Pág}(S))\end{aligned}$$

El precio de no caber en RAM: **3×** el costo del in-memory HJ.

HJ: ejemplo numérico y requisito de memoria

R y S de 16 MB cada una (2.048 páginas), buffer de 128 páginas ($B = 128$).

- **In-memory HJ:** requiere $\text{Pág}(R) \leq 126$. Aquí $\text{Pág}(R) = 2.048 \rightarrow$ **no aplica**.
- **Grace HJ:** Costo = $3 \times (2.048 + 2.048) = 12.288$ I/O $\approx 1,2$ seg

Requisito de memoria para Grace HJ (cada bucket debe caber en RAM):

- Con k buckets balanceados: $\text{Pág}(R)/k \leq B - 2 \rightarrow k \geq \text{Pág}(R)/(B - 2)$
- Además $k \leq B - 1$ (un buffer de escritura por bucket)
- $\rightarrow$ Grace HJ funciona si **$\text{Pág}(R_{\text{menor}}) \leq B^2$** (regla práctica)

HJ: cuándo conviene

- Tablas **grandes** sin índice → **HJ es generalmente el más rápido** para equi-joins
- **No requiere** que el resultado esté ordenado
- **Solo equi-joins** ($R.a = S.a$) — no funciona con desigualdades
- Si $\text{Pág}(R_{\text{menor}}) > B^2$, hay que recurrir a **HJ recursivo** (particionar los buckets de nuevo)

Caso	Costo
R cabe en buffer (in-memory HJ)	$\text{Pág}(R) + \text{Pág}(S)$
$\text{Pág}(R_{\text{menor}}) \leq B^2$ (Grace HJ)	$3 \cdot (\text{Pág}(R) + \text{Pág}(S))$

Comparación y Resumen

Algoritmo	Mejor caso	Peor caso	Requisitos
NLJ	—	$\text{Pág}(R) + \text{Tup}(R) \cdot \text{Pág}(S)$	ninguno
BNLJ	$\text{Pág}(R) + \text{Pág}(S)$ (R cabe en buffer)	$\text{Pág}(R) + \lceil \text{Pág}(R)/(B-2) \rceil \cdot \text{Pág}(S)$	buffer grande ayuda
INLJ	$\text{Pág}(R) + \text{Tup}(R) \cdot 1$ (clustered)	$\text{Pág}(R) + \text{Tup}(R) \cdot (\text{prof} + \text{matches})$	índice en S
SMJ	$\text{Pág}(R) + \text{Pág}(S)$ (ya ordenadas)	$\sim 5 \cdot (\text{Pág}(R) + \text{Pág}(S))$	—
HJ	$\text{Pág}(R) + \text{Pág}(S)$ (in-memory)	$3 \cdot (\text{Pág}(R) + \text{Pág}(S))$ (Grace)	equi-join; $\text{Pág}(R_{\text{menor}}) \leq B^2$

¿Cuándo usar cada algoritmo?

Situación	Mejor algoritmo
Tablas pequeñas, una cabe en buffer	BNLJ
Hay un índice en el atributo de join de S	INLJ
Tablas grandes, resultado debe estar ordenado	SMJ
Tablas grandes, solo equi-join	HJ (generalmente el más rápido)
Join de desigualdad ($>$, $<$, $\neq$)	BNLJ o SMJ
Una tabla ya ordenada por el atributo de join	SMJ (se ahorra un Sort)

Y si la consulta tiene más de un join?

```
SELECT ...  
FROM R, S, T  
WHERE R.a = S.a AND S.b = T.b
```

- El optimizador debe decidir el **orden** de los joins
- Y **qué algoritmo** usar para cada uno
- Esto es parte de la **planificación de consultas** → próxima clase

Resumen

- **Sort-Merge Join:** ordena ambas tablas, luego merge lineal $\rightarrow \sim 5 \cdot (\text{Pág}(R) + \text{Pág}(S))$
- **Hash Join:** in-memory si la chica cabe $(\text{Pág}(R) + \text{Pág}(S))$; Grace si no $(3 \cdot (\text{Pág}(R) + \text{Pág}(S)))$

Ya conocemos 5 algoritmos de join — cada uno con sus fortalezas.

Ejercicio: SMJ y HJ

Tablas: Pedidos con 10.000 páginas (~500.000 tuplas) y Clientes con 500 páginas (~25.000 tuplas). Buffer de 200 páginas. B+ Tree de profundidad 3 en Clientes.cid.

```
SELECT * FROM Pedidos P JOIN Clientes C ON P.cid = C.cid
```

1. Calcule el costo con **BNLJ** (¿cuál tabla como outer?)
2. Calcule el costo con **INLJ** (usando el índice en Clientes)
3. Calcule el costo con **SMJ** (sin pipelining)
4. Calcule el costo con **Hash Join** — ¿aplica in-memory o hay que usar Grace?
5. ¿Cuál algoritmo gana? ¿Cambiaría si Clientes tuviera 5.000 páginas?

Ejercicio propuesto

```
SELECT C.nombre, P.fecha  
FROM Clientes C JOIN Pedidos P ON C.cid = P.cid  
WHERE C.ciudad = 'Santiago' AND P.monto > 100000
```

Suponga que el filtro `ciudad = 'Santiago'` reduce Clientes a 100 páginas, y `monto > 100000` reduce Pedidos a 2.000 páginas.

1. Dibuje el plan lógico (árbol) y aplique push-down de selecciones
2. ¿Qué algoritmo de join conviene para las tablas filtradas?
3. ¿Qué índices propondría para optimizar esta consulta?